Problem Statement

Design a fraud detection system to identify fraudulent credit card transactions in real-time. The system should detect suspicious activities such as unauthorized transactions, unusual spending patterns, and fraudulent card usage to prevent financial losses for both the cardholders and financial institutions.

```
# Load library package
import pandas as pd  # Organise and manipulate the data
import numpy as np   # Scientifical computation package
import matplotlib.pyplot as plt # Visualization
import seaborn as sns           # Visulaization
import warnings
warnings.filterwarnings('ignore')
```

```
# Load the data
data = pd.read_csv('cc_data.csv')
print("==="*10)
print("data has successfully loaded..")
```

```
==============================
data has successfully loaded..
```

```
data.head()
```

	index	trans_date_trans_time	cc_num	merchant	category
0	151888	23/03/19 3:06	3.586008e+15	fraud_Ferry, Lynch and Kautzer	misc_net
1	1185025	10/05/20 11:28	4.671727e+15	fraud_Bogisich Inc	grocery_pos
2	10818	07/01/19 13:31	3.779930e+14	fraud_Mohr-Bayer	shopping_net
3	975275	30/01/20 18:49	4.710826e+18	fraud_Langworth LLC	personal_care
4	346814	12/06/19 3:04	3.723820e+14	fraud_Hills-Witting	shopping_net

5 rows × 23 columns

```
data.tail(2)
```

	index	trans_date_trans_time	cc_num	merchant	categ
28460	583949	04/09/19 18:17	4.951647e+15	fraud_Walter, Hettinger and Kessler	personal_c
28461	1272527	12/06/20 19:24	2.131420e+14	fraud_Bechtelar-Rippin	food_dir

2 rows × 23 columns

```
# Summary of dataframe
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28462 entries, 0 to 28461
Data columns (total 23 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   index                  28462 non-null  int64
 1   trans_date_trans_time  28462 non-null  object
 2   cc_num                 28462 non-null  float64
 3   merchant               28462 non-null  object
 4   category               28462 non-null  object
 5   amt                    28462 non-null  float64
 6   first                  28462 non-null  object
 7   last                   28462 non-null  object
 8   gender                 28462 non-null  object
 9   street                 28462 non-null  object
 10  city                   28462 non-null  object
 11  state                  28462 non-null  object
 12  zip                    28462 non-null  int64
 13  lat                    28462 non-null  float64
 14  long                   28462 non-null  float64
 15  city_pop               28462 non-null  int64
 16  job                    28462 non-null  object
 17  dob                    28462 non-null  object
 18  trans_num              28462 non-null  object
 19  unix_time              28462 non-null  int64
 20  merch_lat              28462 non-null  float64
 21  merch_long             28461 non-null  float64
 22  is_fraud               28461 non-null  float64
dtypes: float64(7), int64(4), object(12)
memory usage: 5.0+ MB
```

Task 1: What are the dimensions (number of rows and columns) of the dataset?

```
# Knowing the dimension of dataframe
print("The dimension of dataset is:", data.shape)
```

```
The dimension of dataset is: (28462, 23)
```

Task 2: How many unique values are there in each categorical variable?

```
categorical_columns = data.select_dtypes(include=['object']).columns
print(categorical_columns)
```

```
Index(['trans_date_trans_time', 'merchant', 'category', 'first', 'last',
       'gender', 'street', 'city', 'state', 'job', 'dob', 'trans_num'],
      dtype='object')
```

```
for col in categorical_columns:
  print(f"{col} has unique values : {data[col].nunique()}")
```

```
trans_date_trans_time has unique values : 27839
merchant has unique values : 693
category has unique values : 14
first has unique values : 341
last has unique values : 468
gender has unique values : 2
street has unique values : 927
city has unique values : 851
state has unique values : 50
job has unique values : 482
dob has unique values : 912
trans_num has unique values : 28462
```

Task 3: What is the distribution of numerical variables in the dataset?

```
# Based on domain knowledge and understanding, let take amt and city_pop var
num_data = data[['amt', 'city_pop']].copy()
num_data.head(3)
```

	amt	city_pop
0	1.24	50489
1	112.57	2304

2	6.69	5548

```
# Distribution of the columns
num_data.describe()
```

	amt	city_pop
count	28462.000000	2.846200e+04
mean	72.352412	9.103926e+04
std	205.419209	3.043625e+05
min	1.000000	2.300000e+01
25%	9.670000	7.430000e+02
50%	47.945000	2.435000e+03
75%	83.200000	2.032800e+04
max	15305.950000	2.906700e+06

Findings:

1. amt is a right skewed distribution since the mean is greater than the median of data.
2. City_pop is also a right skewed distribution.

Task 4: Are there any missing values in the dataset? If so, how should they be handled?

```
# Knowing missing values
data.isnull().sum()
```

	0
index	0
trans_date_trans_time	0
cc_num	0
merchant	0
category	0
amt	0

amt	0
first	0
last	0
gender	0
street	0
city	0
state	0
zip	0
lat	0
long	0
city_pop	0
job	0
dob	0
trans_num	0
unix_time	0
merch_lat	0
merch_long	1
is_fraud	1

dtype: int64

Handling of missing values:

1. For categorical columns use mode as imputation value
2. For numerical columns use mean (when the distribution is symmetric) or median(when distribution is skewed) to impute missing values

Task 5: What are the summary statistics (mean, median, min, max, etc.) for numerical variables?

```
# Distribution of the columns
num_data.describe()
```

	amt	city_pop

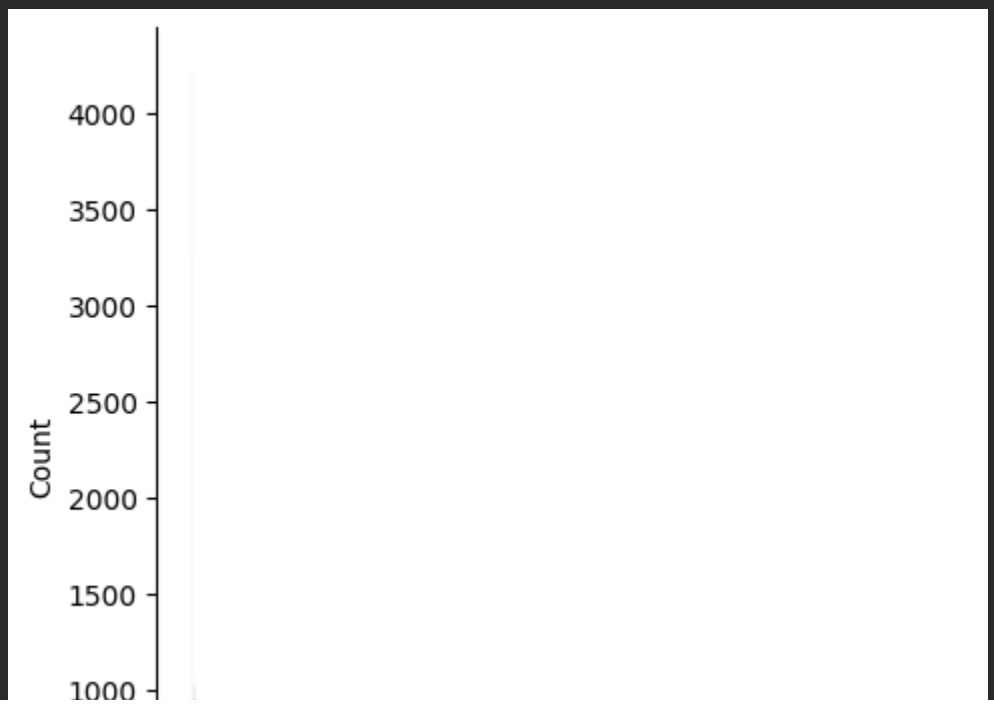
count	28462.000000	2.846200e+04
mean	72.352412	9.103926e+04
std	205.419209	3.043625e+05
min	1.000000	2.300000e+01
25%	9.670000	7.430000e+02
50%	47.945000	2.435000e+03
75%	83.200000	2.032800e+04
max	15305.950000	2.906700e+06

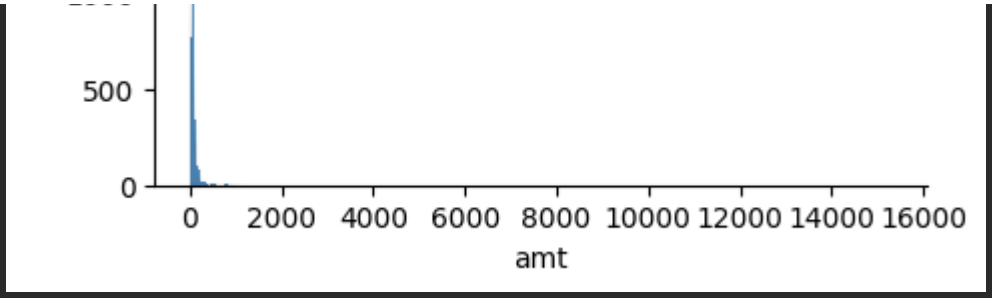
Task 6: Is there any correlation between numerical variables? If so, how strong is the correlation?

```
num_data.corr()
```

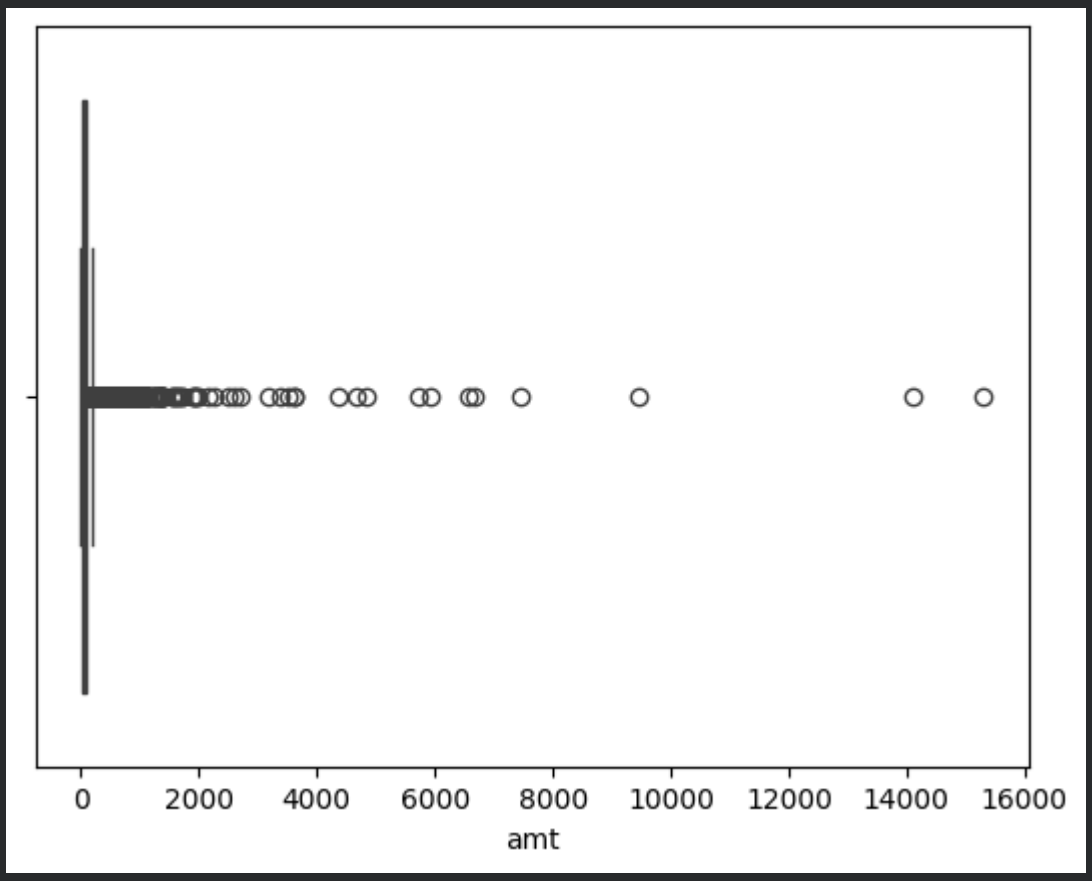
	amt	city_pop
amt	1.00000	0.00541
city_pop	0.00541	1.00000

```
# Draw the distibution of numerical variables
sns.displot(data= num_data, x = 'amt')
plt.show()
```

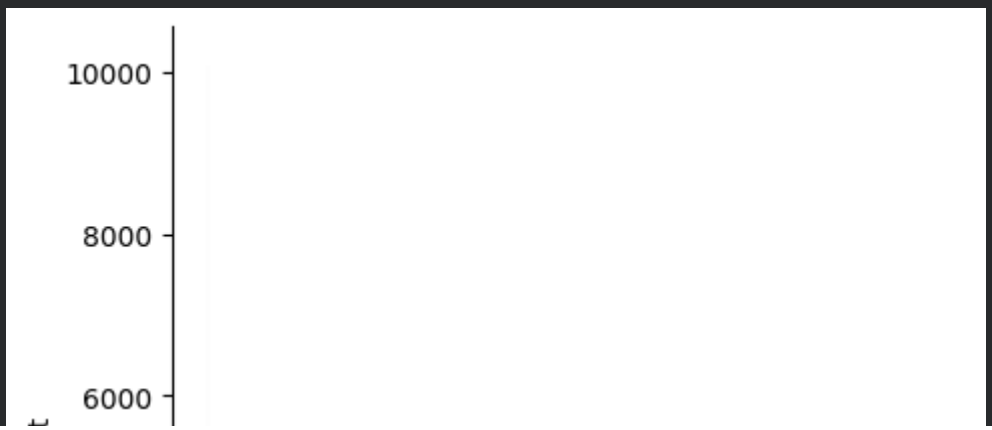


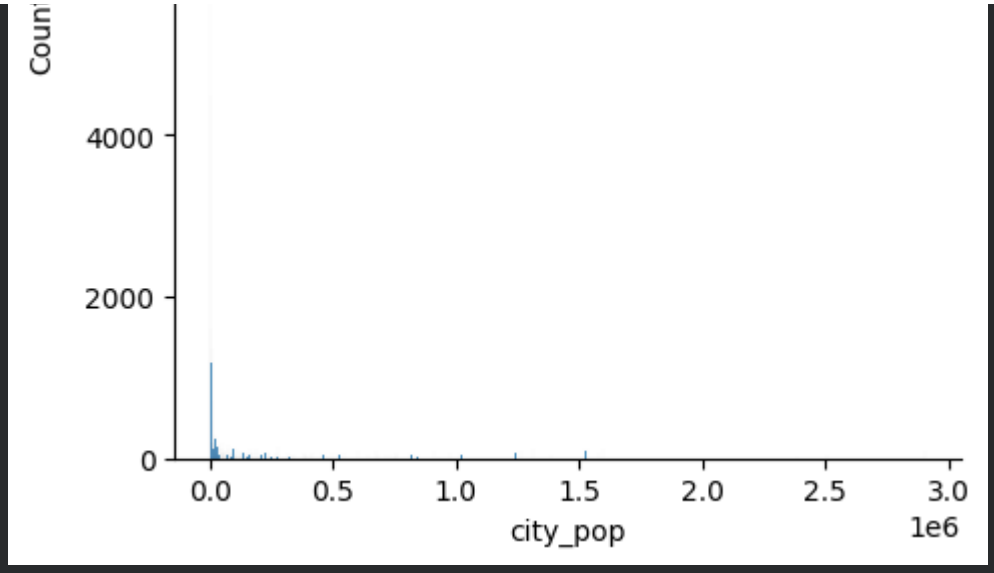



```
sns.boxplot(data= num_data, x = 'amt')
plt.show()
```

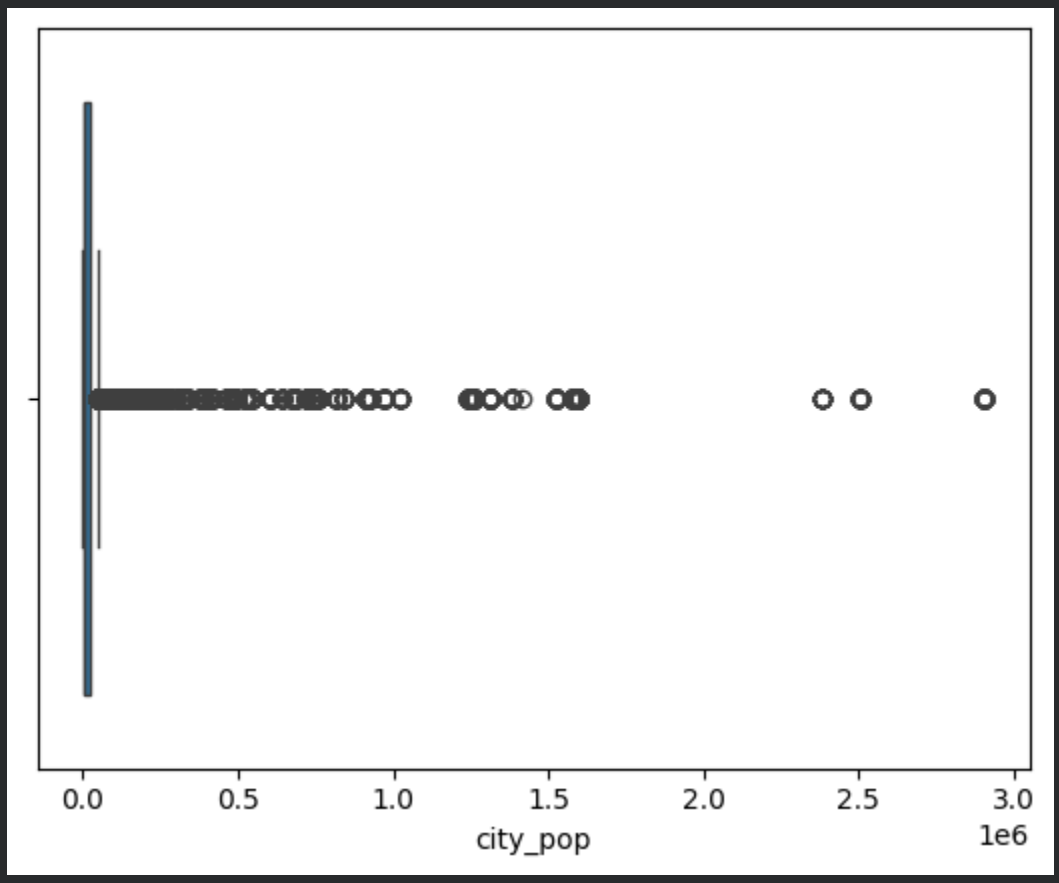



```
sns.displot(data= num_data, x = 'city_pop')
plt.show()
```





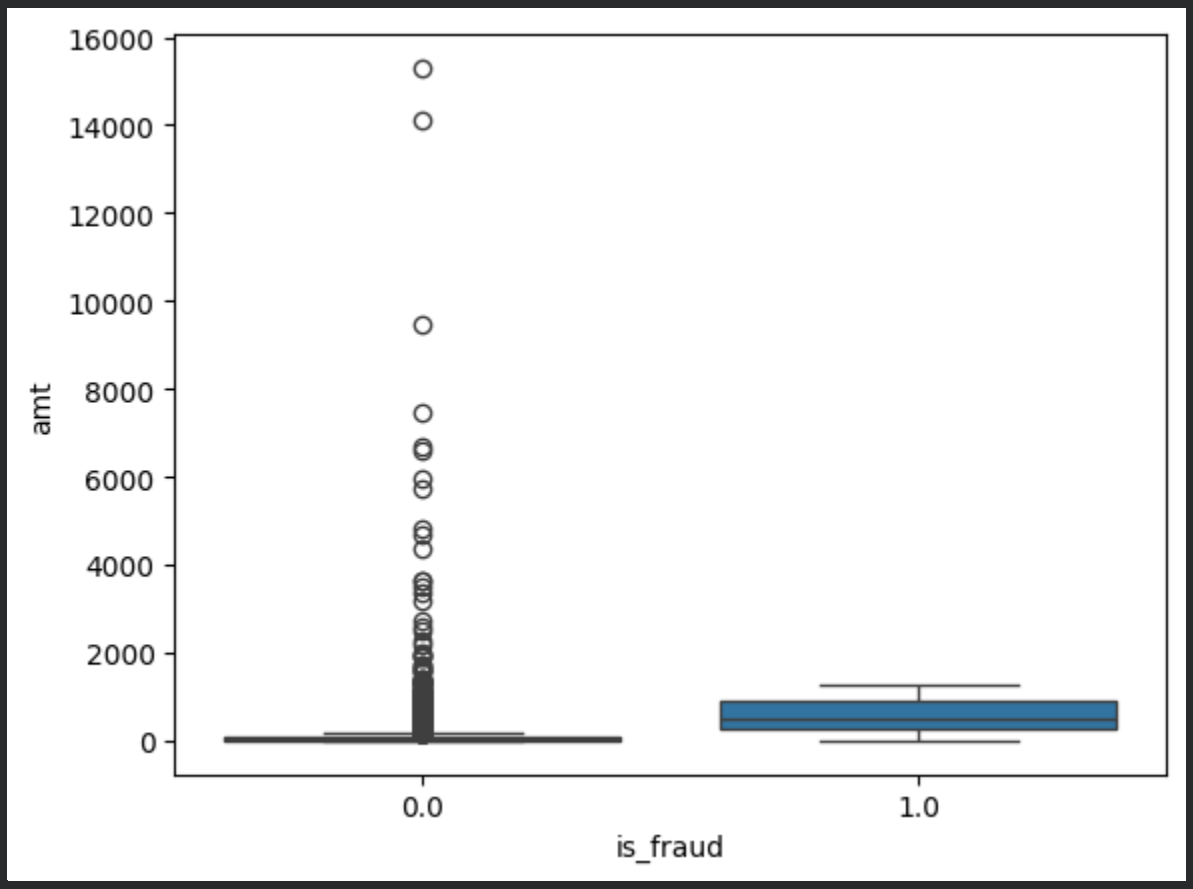

```
sns.boxplot(data= num_data, x = 'city_pop')
plt.show()
```



Task 7: How does the distribution of an amt differ across is_fraud categories?

```
sns.boxplot(data= data, x = 'is_fraud', y ='amt')
plt.show()
```

```
plt.show()
```



Task 8: Are there any outliers in the city_pop and amt?

```
# Number of outliers in amt column
Q1 = data['amt'].quantile(0.25)
Q3 = data['amt'].quantile(0.75)
IQR = Q3 - Q1
# Rule of outlier: x < Q1-1.5*IQR or x > Q3+1.5*IQR is outlier
lower_bound = Q1 - 1.5*IQR
upper_bound = Q3 + 1.5*IQR
amt_outliers = data[(data['amt'] < lower_bound) | (data['amt'] > upper_bound)]
print("Number of outliers in amt column:", len(amt_outliers))
```

```
Number of outliers in amt column: 1477
```

Task 9: Are there any trends or patterns in the data over time (if applicable)?

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28462 entries, 0 to 28461
```

```
Data columns (total 23 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   index                  28462 non-null  int64
 1   trans_date_trans_time  28462 non-null  object
 2   cc_num                 28462 non-null  float64
 3   merchant               28462 non-null  object
 4   category               28462 non-null  object
 5   amt                    28462 non-null  float64
 6   first                  28462 non-null  object
 7   last                   28462 non-null  object
 8   gender                 28462 non-null  object
 9   street                 28462 non-null  object
 10  city                   28462 non-null  object
 11  state                  28462 non-null  object
 12  zip                    28462 non-null  int64
 13  lat                    28462 non-null  float64
 14  long                   28462 non-null  float64
 15  city_pop               28462 non-null  int64
 16  job                    28462 non-null  object
 17  dob                    28462 non-null  object
 18  trans_num              28462 non-null  object
 19  unix_time              28462 non-null  int64
 20  merch_lat              28462 non-null  float64
 21  merch_long             28461 non-null  float64
 22  is_fraud               28461 non-null  float64
dtypes: float64(7), int64(4), object(12)
memory usage: 5.0+ MB
```

```
# Convert the trans_date_trans_time column into datetime column type
data['trans_date_trans_time'] = pd.to_datetime(data['trans_date_trans_time']
data.info()
```

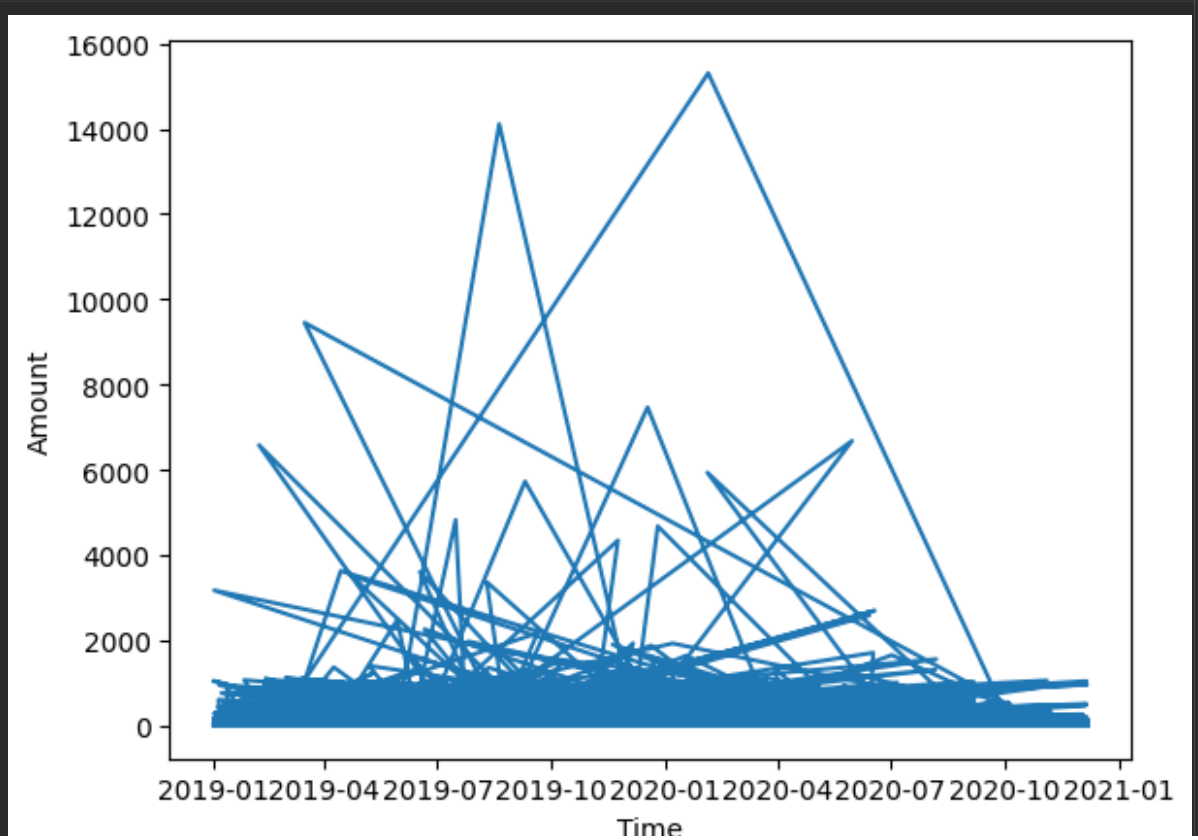
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28462 entries, 0 to 28461
Data columns (total 23 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   index                  28462 non-null  int64
 1   trans_date_trans_time  28462 non-null  datetime64[ns]
 2   cc_num                 28462 non-null  float64
 3   merchant               28462 non-null  object
 4   category               28462 non-null  object
 5   amt                    28462 non-null  float64
 6   first                  28462 non-null  object
 7   last                   28462 non-null  object
 8   gender                 28462 non-null  object
 9   street                 28462 non-null  object
 10  city                   28462 non-null  object
 11  state                  28462 non-null  object
 12  zip                    28462 non-null  int64
 13  lat                    28462 non-null  float64
 14  long                   28462 non-null  float64
 15  city_pop               28462 non-null  int64
```

```
 16  job                     28462 non-null  object
 17  dob                     28462 non-null  object
 18  trans_num               28462 non-null  object
 19  unix_time               28462 non-null  int64
 20  merch_lat               28462 non-null  float64
 21  merch_long              28461 non-null  float64
 22  is_fraud                28461 non-null  float64
dtypes: datetime64[ns](1), float64(7), int64(4), object(11)
memory usage: 5.0+ MB
```

```
time_data= data[['trans_date_trans_time', 'amt']].copy()
time_data.head(5)
```

	trans_date_trans_time	amt
0	2019-03-23 03:06:00	1.24
1	2020-10-05 11:28:00	112.57
2	2019-07-01 13:31:00	6.69
3	2020-01-30 18:49:00	100.69
4	2019-12-06 03:04:00	7.80

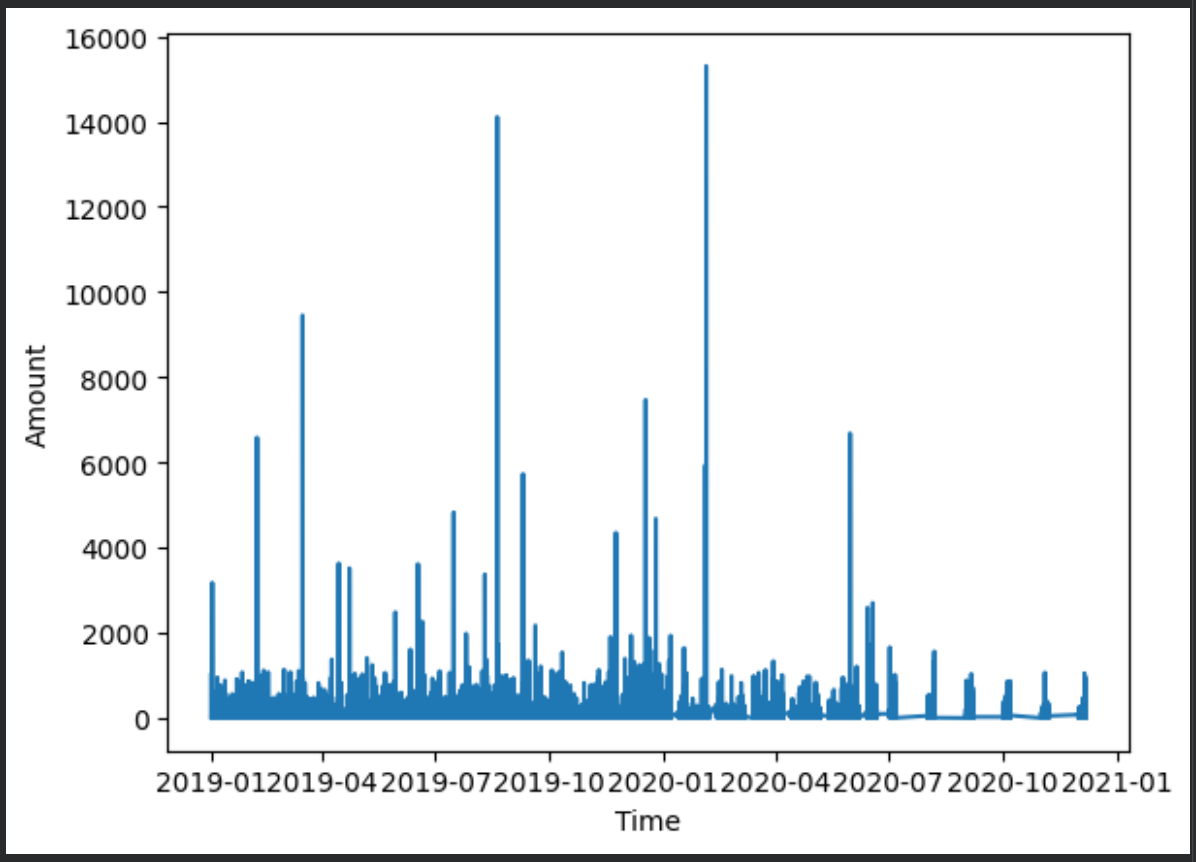
```
plt.plot(time_data['trans_date_trans_time'], time_data['amt'])
plt.xlabel('Time')
plt.ylabel('Amount')
plt.show()
```



```
# Sort the time_data using date column in ascending order
time_data = time_data.sort_values(by='trans_date_trans_time')
time_data.head()
```

	trans_date_trans_time	amt
4923	2019-01-01 00:22:00	3.66
26521	2019-01-01 01:39:00	68.24
15585	2019-01-01 02:54:00	1055.47
7426	2019-01-01 04:37:00	2.73
22007	2019-01-01 06:10:00	1.34

```
plt.plot(time_data['trans_date_trans_time'], time_data['amt'])
plt.xlabel('Time')
plt.ylabel('Amount')
plt.show()
```



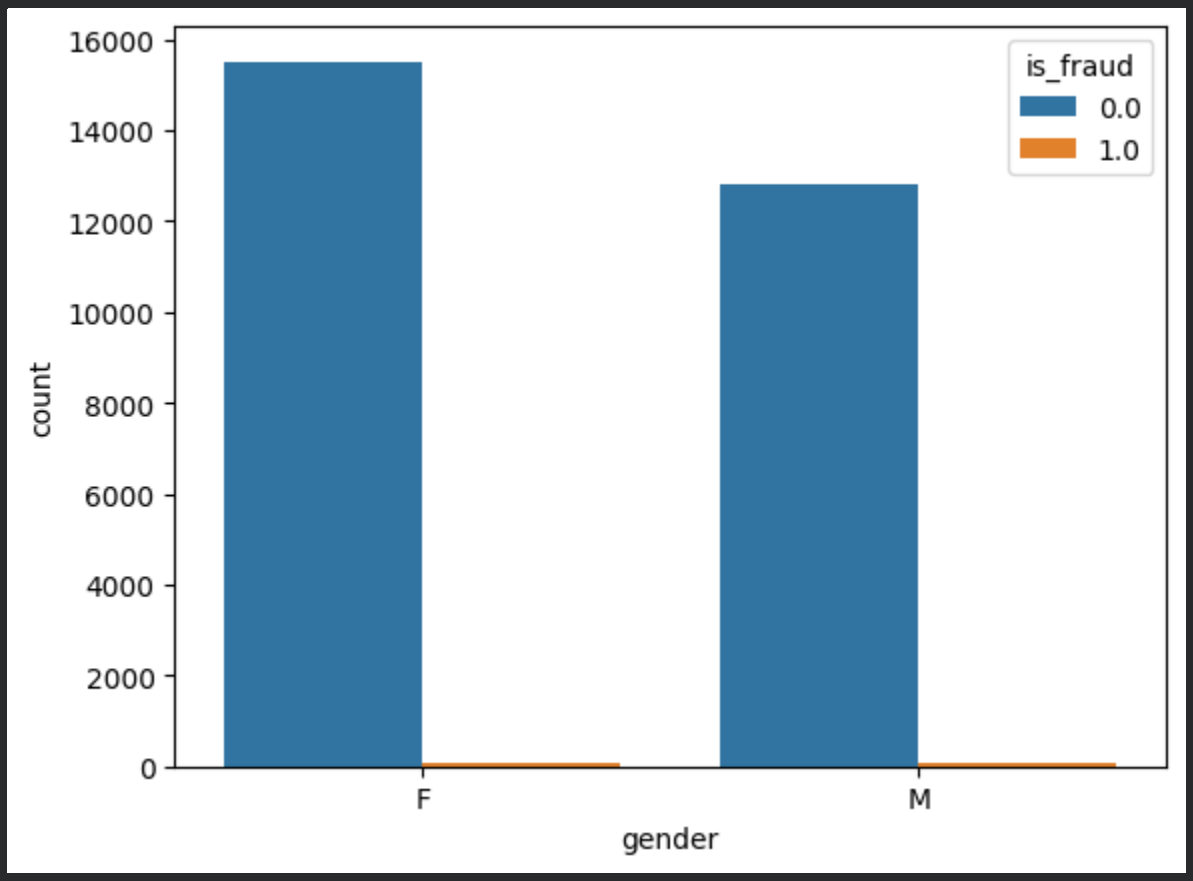
Task 10: How does the target variable (if available) distribute across different categories?

different categories?

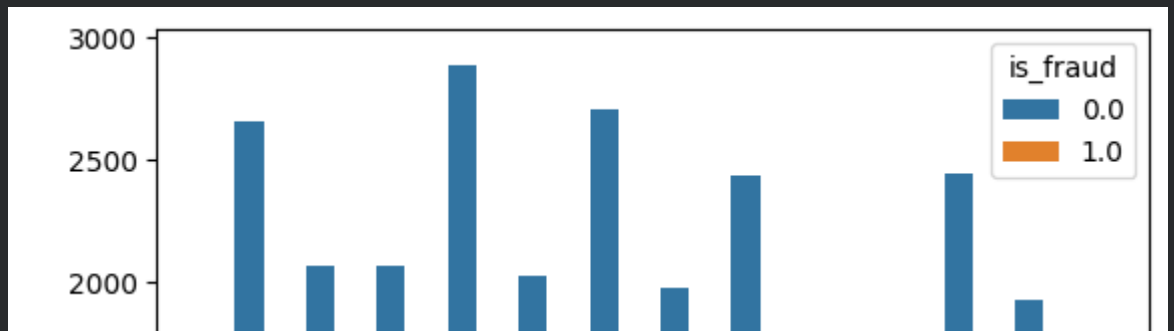
```
data.columns
```

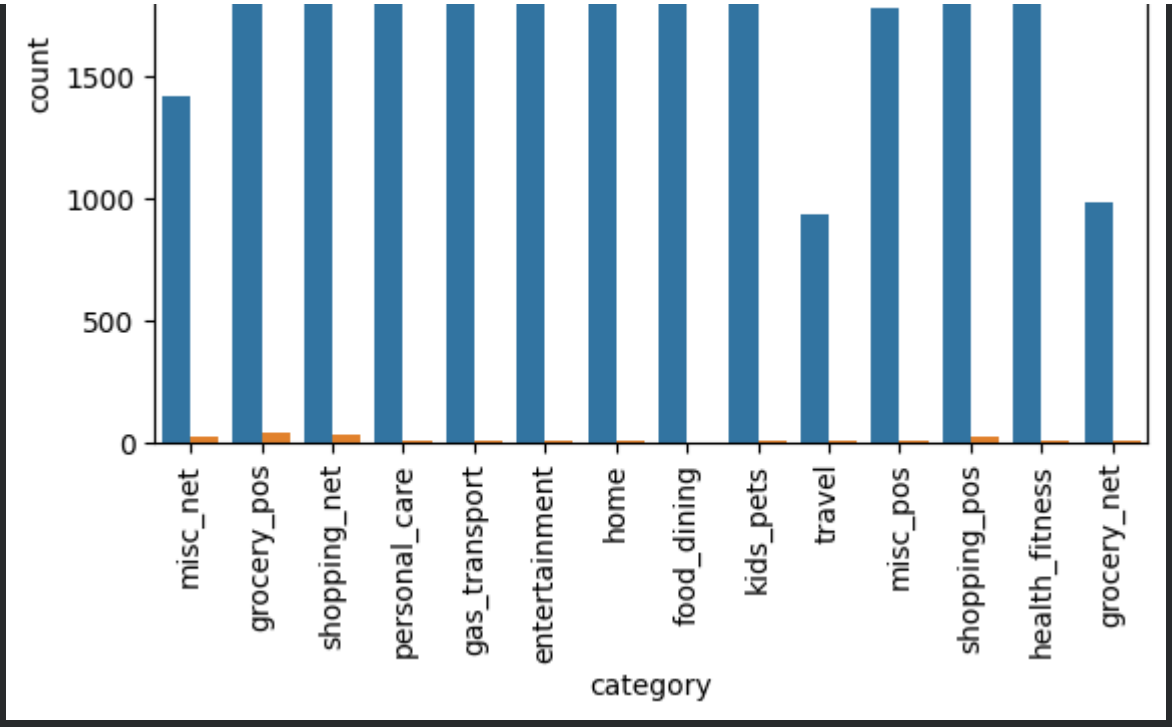
```
Index(['index', 'trans_date_trans_time', 'cc_num', 'merchant', 'category',
       'amt', 'first', 'last', 'gender', 'street', 'city', 'state', 'zip',
       'lat', 'long', 'city_pop', 'job', 'dob', 'trans_num', 'unix_time',
       'merch_lat', 'merch_long', 'is_fraud'],
      dtype='object')
```

```
sns.countplot(data= data, x= 'gender', hue = 'is_fraud')
plt.show()
```

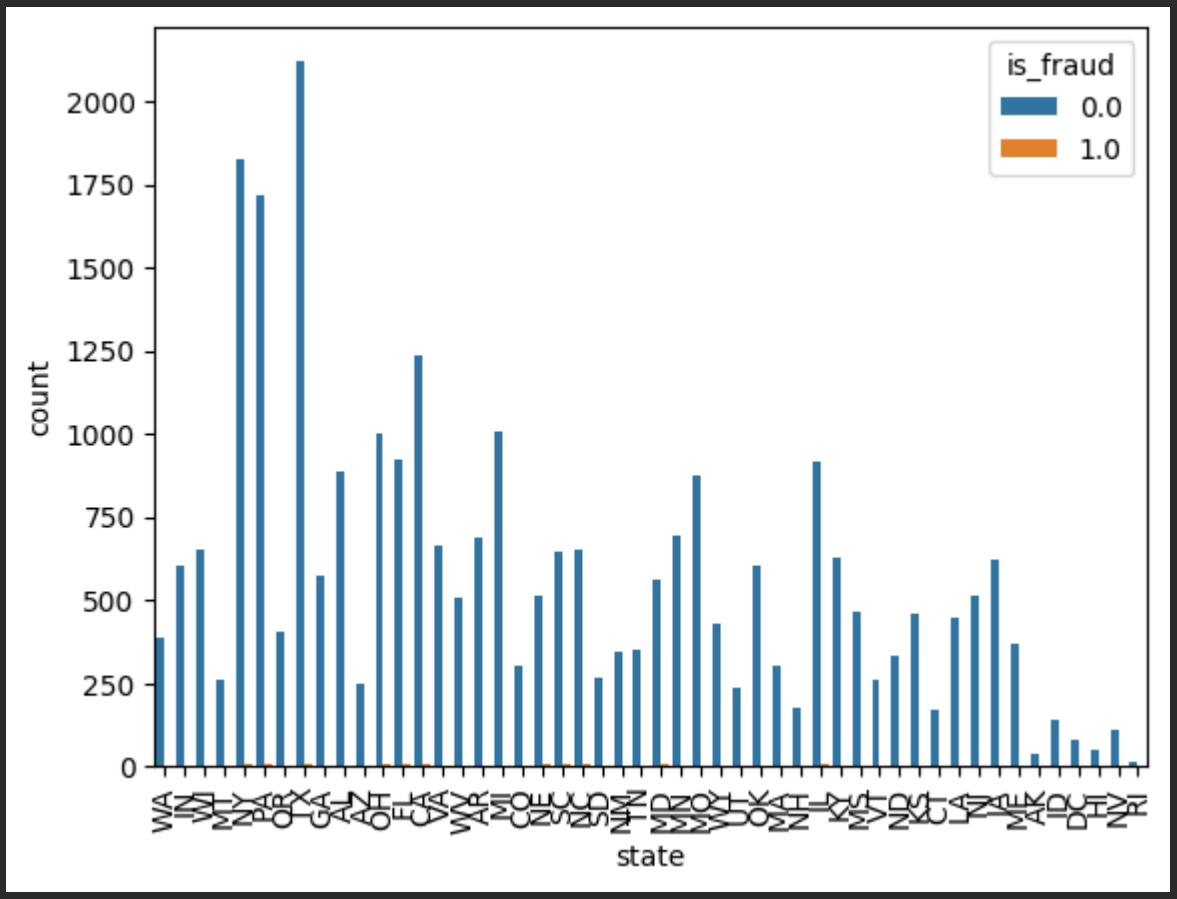



```
sns.countplot(data= data, x= 'category', hue = 'is_fraud')
plt.xticks(rotation=90)
plt.show()
```



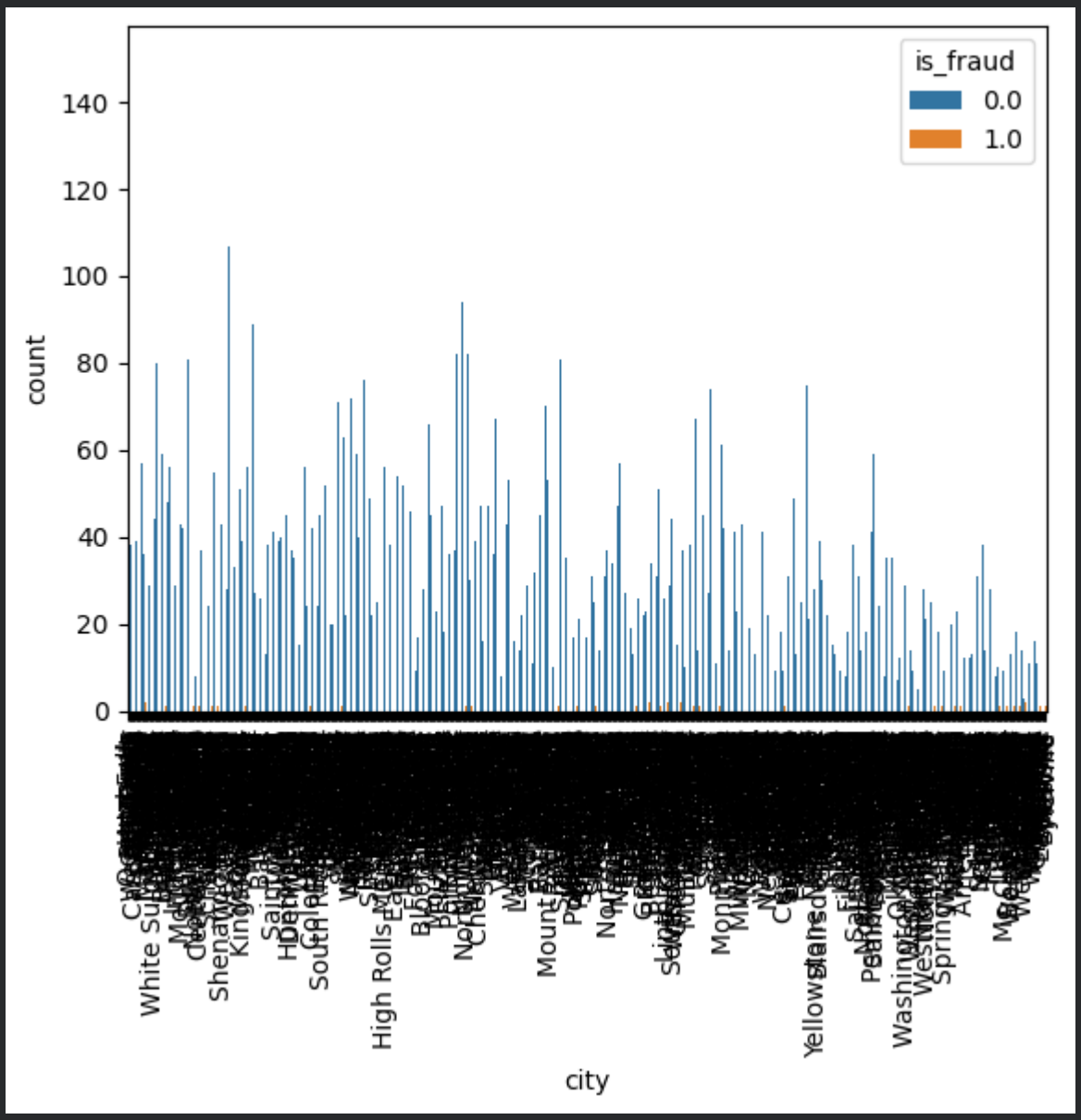



```
sns.countplot(data= data, x= 'state', hue = 'is_fraud')
plt.xticks(rotation=90)
plt.show()
```




```
sns.countplot(data= data, x= 'city', hue = 'is_fraud')
```

```
plt.xticks(rotation=90)
plt.show()
```



Task 11: Are there any unusual or unexpected values in the dataset that require further investigation?

```
data.columns
```

```
Index(['index', 'trans_date_trans_time', 'cc_num', 'merchant', 'category',
       'amt', 'first', 'last', 'gender', 'street', 'city', 'state', 'zip',
       'lat', 'long', 'city_pop', 'job', 'dob', 'trans_num', 'unix_time',
       'merch_lat', 'merch_long', 'is_fraud'],
      dtype='object')
```

```
data['amt'].describe()
```

	amt
count	28462.000000
mean	72.352412
std	205.419209
min	1.000000
25%	9.670000
50%	47.945000
75%	83.200000
max	15305.950000

dtype: float64

```
data['city_pop'].describe()
```

	city_pop
count	2.846200e+04
mean	9.103926e+04
std	3.043625e+05
min	2.300000e+01
25%	7.430000e+02
50%	2.435000e+03
75%	2.032800e+04
max	2.906700e+06

dtype: float64

```
data['gender'].unique()
```

```
array(['F', 'M'], dtype=object)
```

```
data['category'].unique()
```

```
array(['misc_net', 'grocery_pos', 'shopping_net', 'personal_care',
       'gas_transport', 'entertainment', 'home', 'food_dining',
       'kids_pets', 'travel', 'misc_pos', 'shopping_pos',
       'health_fitness', 'grocery_net'], dtype=object)
```

```
data['job'].unique()
```

```
array(['Osteopath', 'Retail banker', 'Mining engineer',
       'Further education lecturer', 'Heritage manager', 'Archaeologist',
       'Film/video editor', 'Barrister', 'Psychiatrist',
       'Education officer, museum', 'Pensions consultant',
       'Pension scheme manager', 'Chemical engineer',
       'Petroleum engineer', 'Child psychotherapist',
       'Magazine features editor', 'Environmental health practitioner',
       'Exercise physiologist', 'Paediatric nurse',
       'Multimedia programmer', 'Buyer, industrial',
       'Designer, multimedia', 'Animator', 'Community development worker',
       'Nutritional therapist', 'Careers information officer',
       'Learning disability nurse', 'Engineer, electronics',
       'Television/film/video producer', 'Research scientist (maths)',
       'Biomedical engineer', 'Statistician', 'Chief Strategy Officer',
       'Hospital pharmacist', 'Charity officer', 'Journalist, newspaper',
       'Purchasing manager', 'Music tutor', 'Town planner',
       'Health and safety adviser', 'Engineer, production',
       'Biochemist, clinical', 'Therapist, horticultural', 'Chiropodist',
       'Chartered loss adjuster', 'Lecturer, higher education',
       'Press sub', 'Geologist, wellsite', 'Therapist, music',
       'Telecommunications researcher', 'Surveyor, minerals',
       'Freight forwarder', 'Special educational needs teacher',
       'Fitness centre manager', 'Energy engineer', 'Seismic interpreter',
       'Glass blower/designer', 'Farm manager', 'Naval architect',
       'Dispensing optician', 'Psychologist, counselling',
       'Market researcher', 'Optician, dispensing', 'Exhibition designer',
       'Sub', 'Site engineer', 'Conservation officer, historic buildings',
       'Equality and diversity officer', 'Chief Executive Officer',
       'Gaffer',
       'Scientist, clinical (histocompatibility and immunogenetics)',
       'Call centre manager', 'Manufacturing engineer', 'Land',
       'Claims inspector/assessor', 'Hospital doctor',
       'Copywriter, advertising', 'Control and instrumentation engineer',
       'Ceramics designer', 'Public affairs consultant',
       'Designer, ceramics/pottery', 'Early years teacher',
       'Product/process development scientist', 'IT trainer',
       'Systems developer', 'Corporate investment banker',
       'Landscape architect', 'Prison officer', 'Teacher, primary school',
       'Media buyer', 'Research scientist (physical sciences)',
       'Radio producer', 'Production manager', 'Paramedic',
       'Garment/textile technologist', 'Physiological scientist',
       'Systems analyst', 'Health service manager', 'Records manager',
       'Curator', 'Insurance broker', 'Producer, radio', 'Therapist, art',
       'Psychologist, forensic', 'Research scientist (life sciences)',
       'Immigration officer',
       'Historic buildings inspector/conservation officer',
       'Community education officer', 'Editor, commissioning',
       'Electrical engineer', 'Operational researcher',
       'Volunteer coordinator', 'Aid worker',
       'Exhibitions officer, museum/gallery',
       'Engineer, civil (contracting)', 'Land/geomatics surveyor',
       'Musician', 'Metallurgist', 'Advertising account planner',
       'Geochemist', 'Trade mark attorney', 'Teacher, early years/pre',
```

```
       'Counsellor', 'Museum/gallery exhibitions officer',
       'Chief Technology Officer', 'Quantity surveyor',
       'Leisure centre manager', 'Teaching laboratory technician',
       'Geologist, engineering', 'Technical brewer',
       'Sport and exercise psychologist', 'Building control surveyor',
```

```
data['job'].nunique()
```

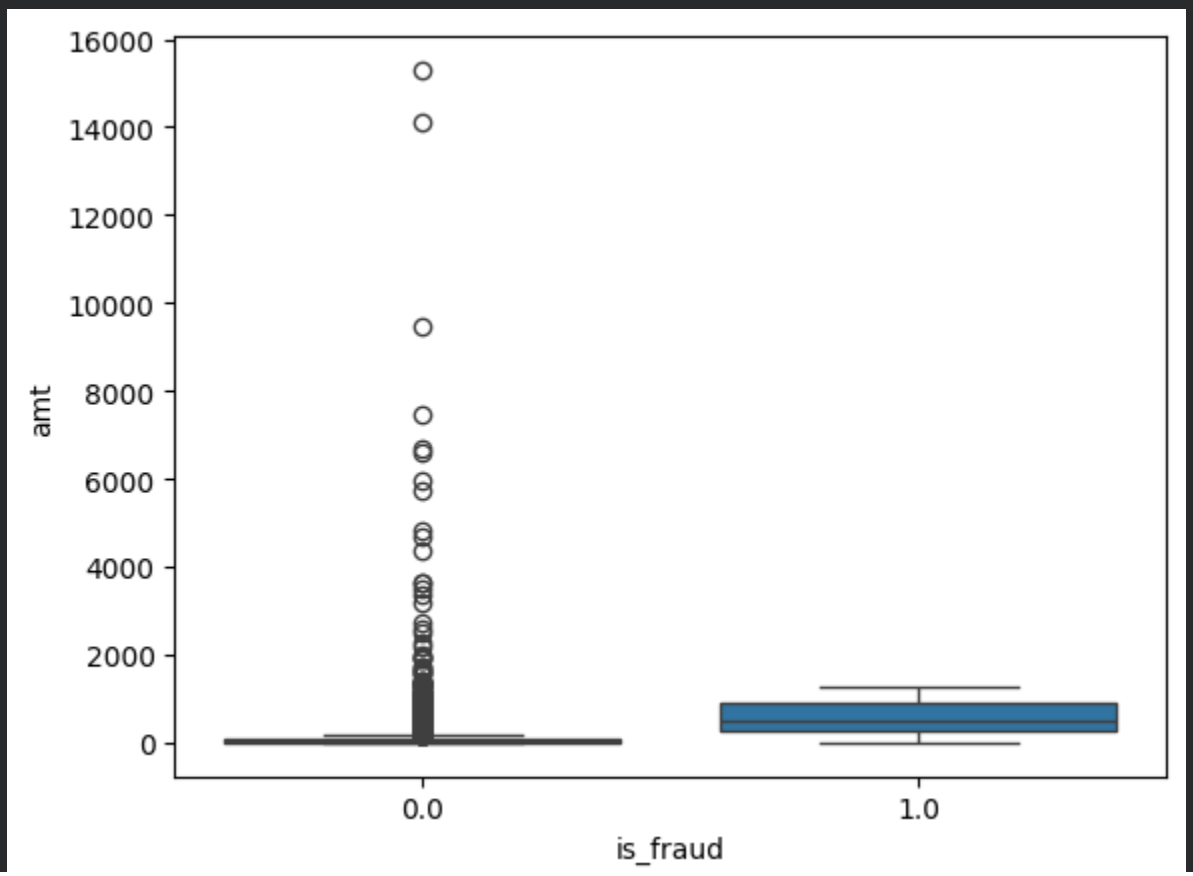
```
482
```

Task 12: Are there any potential data entry errors or inconsistencies in the dataset?

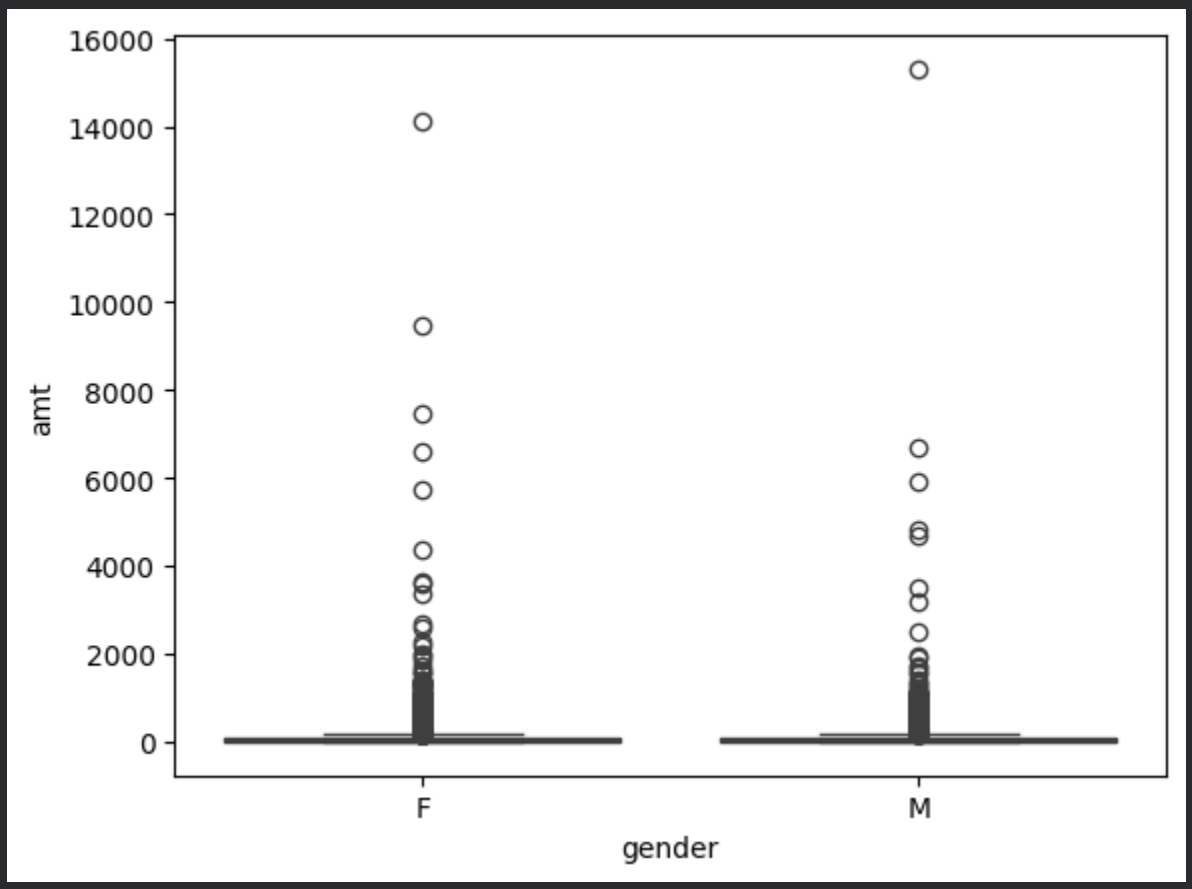
```
# NO
```

Task 13: How does the distribution of numerical variables vary between different groups or segments of the dataset?

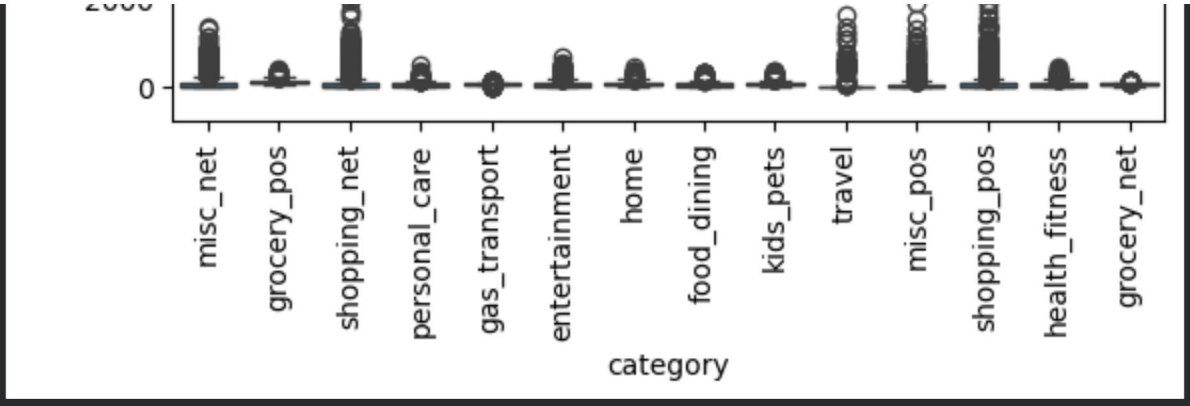
```
# Numerical variables- amt and city_pop
# Segments: Categorical Varibale- is_fruad(target variable), Gender, State,
sns.boxplot(data= data, x= 'is_fraud', y= 'amt')
plt.show()
```



```
sns.boxplot(data= data, x= 'gender', y= 'amt')
plt.show()
```




```
sns.boxplot(data= data, x= 'category', y= 'amt')
plt.xticks(rotation=90)
plt.show()
```



Start coding or generate with AI.